

Міністерство освіти і науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

**з дисципліни «Об'єктно-орієнтоване
програмування»**

на тему

**«Багатовіконний інтерфейс, Singleton та
збереження даних»**

Виконав:

Мащута Олександр
студент групи ІМ-051
номер у списку групи: 7

Перевірів:

Рекечинський Дмитро Олександрович

Київ 2026

Завдання

1. Перетворити клас `MyEditor` на патерн **Singleton Меєрса** (потокобезпечна лінива ініціалізація).
2. Створити немодальне вікно таблиці `MyTableForm` як незалежний модуль.
3. Забезпечити автоматичне оновлення таблиці при створенні/завантаженні об'єктів.
4. Реалізувати збереження списку створених об'єктів у текстовий файл та їх завантаження з файлу.
5. Налаштувати багатовіконну взаємодію у графічному середовищі C# WinForms.
6. Оформити звіт.

Завдання згідно варіанту

Для студента №7 (Ж = 7):

1. **Патерн Singleton Меєрса:** Реалізований через `Lazy<MyEditor>` у режимі `ExecutionAndPublication` (C#-еквівалент `static MyEditor instance` у C++), що забезпечує гарантовану єдиність екземпляра та потокобезпеку.
2. **Немодальне вікно таблиці (`MyTableForm`):** Містить `ListView` з колонками `Name`, `X1`, `Y1`, `X2`, `Y2`. Форма приймає прості кортежі даних і не має жорсткої залежності від типів `Shape` чи `MyEditor`.
3. **Робота з файлами:** Формат збереження — TSV (Tab-Separated Values). Завантаження відновлює об'єкти за іменем класу через фабричний метод `CreateShapeByName`.

Вихідний текст програми

Клас `Singleton MyEditor.cs`

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Globalization;
using System.IO;
using Lab5.Shapes;

namespace Lab5;

public class MyEditor
{
    private static readonly Lazy<MyEditor> _lazy = new(() => new MyEditor());
    public static MyEditor Instance => _lazy.Value;

    private readonly object _sync = new();
    private readonly List<Shape> _shapes = [];

    private MyEditor() { }

    public void AddShape(Shape shape)
    {
        lock (_sync)
        {
            _shapes.Add(shape);
        }
    }

    /// <summary>
    /// Thread-safe snapshot view of shapes. Prevents race conditions during iteration.
    /// </summary>
    public IReadOnlyList<Shape> GetShapes()
    {
        lock (_sync)
        {
            return _shapes.ToArray();
        }
    }

    public void DrawAll(Graphics g, Pen pen)
    {
        IReadOnlyList<Shape> snapshot = GetShapes();
        foreach (var shape in snapshot)
        {
            shape.Draw(g, pen, GetFillBrush(shape));
        }
    }

    private static Brush GetFillBrush(Shape shape) => shape switch
    {
        RectShape => Brushes.Orange,
        EllipseShape => Brushes.White,
    }
}

```

```

        LineWithCirclesShape => Brushes.Yellow,
        CubeWireframeShape => Brushes.Cyan,
        _ => Brushes.LightGray
    };

    public void SaveToFile(string path)
    {
        using var sw = new StreamWriter(path);
        foreach (var s in GetShapes())
        {
            sw.WriteLine($"{s.GetName()}\t{s.X1}\t{s.Y1}\t{s.X2}\t{s.Y2}");
        }
    }

    public int LoadFromFile(string path)
    {
        var loaded = new List<Shape>();
        Span<Range> ranges = stackalloc Range[6];

        foreach (var line in File.ReadAllLines(path))
        {
            ReadOnlySpan<char> span = line.AsSpan().Trim();
            if (span.IsEmpty) continue;

            int count = span.Split(ranges, '\t', StringSplitOptions.RemoveEmptyEntries);
            if (count < 5) continue;

            string name = span[ranges[0]].ToString();
            if (!int.TryParse(span[ranges[1]], CultureInfo.InvariantCulture, out int x1) ||
                !int.TryParse(span[ranges[2]], CultureInfo.InvariantCulture, out int y1) ||
                !int.TryParse(span[ranges[3]], CultureInfo.InvariantCulture, out int x2) ||
                !int.TryParse(span[ranges[4]], CultureInfo.InvariantCulture, out int y2))
            {
                continue;
            }

            Shape? shape = CreateShapeByName(name, x1, y1, x2, y2);
            if (shape != null) loaded.Add(shape);
        }

        lock (_sync)
        {
            _shapes.Clear();
            _shapes.AddRange(loaded);
        }
        return loaded.Count;
    }

    private static Shape? CreateShapeByName(string name, int x1, int y1, int x2, int y2) =>
    name switch
    {
        {

```

```

        "Point" => new PointShape(x1, y1, x2, y2),
        "Line" => new LineShape(x1, y1, x2, y2),
        "Rectangle" => new RectShape(x1, y1, x2, y2),
        "Ellipse" => new EllipseShape(x1, y1, x2, y2),
        "LineWithCircles" => new LineWithCirclesShape(x1, y1, x2, y2),
        "Cubewireframe" => new CubewireframeShape(x1, y1, x2, y2),
        _ => null
    };
}

```

Модуль вікна таблиці MyTableForm.cs

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;

namespace Lab5
{
    // Незалежний модуль-таблиця (аналог my_table з методички):
    // НЕ залежить від класів Shape та MyEditor – приймає дані у вигляді
    // простих кортежів, тому компонент можна використати в іншому проєкті
    // без жодних залежностей від редактора.
    public class MyTableForm : Form
    {
        private ListView _listView;

        public MyTableForm()
        {
            this.Text = "Shapes Table";
            this.Size = new Size(400, 300);
            this.StartPosition = FormStartPosition.CenterScreen;

            _listView = new ListView
            {
                View = View.Details,
                Dock = DockStyle.Fill,
                FullRowSelect = true
            };

            _listView.Columns.Add("Name", 100);
            _listView.Columns.Add("X1", 50);
            _listView.Columns.Add("Y1", 50);
            _listView.Columns.Add("X2", 50);
            _listView.Columns.Add("Y2", 50);

            this.Controls.Add(_listView);
        }

        public void UpdateData(IEnumerable<(string Name, int X1, int Y1, int X2, int Y2)>
rows)

```

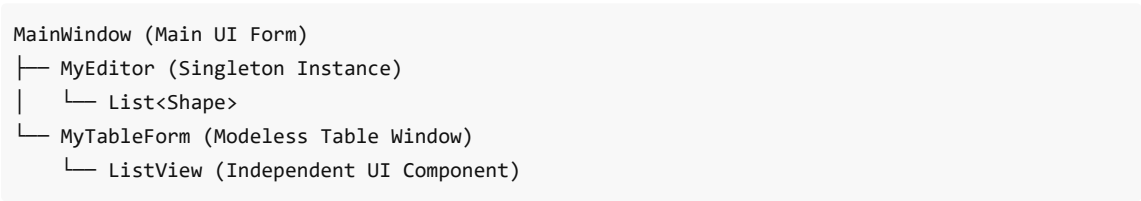
```

    {
        _listView.BeginUpdate();
        _listView.Items.Clear();
        foreach (var (name, x1, y1, x2, y2) in rows)
        {
            var item = new ListViewItem(name);
            item.SubItems.Add(x1.ToString());
            item.SubItems.Add(y1.ToString());
            item.SubItems.Add(x2.ToString());
            item.SubItems.Add(y2.ToString());
            _listView.Items.Add(item);
        }
        _listView.EndUpdate();
    }
}
}

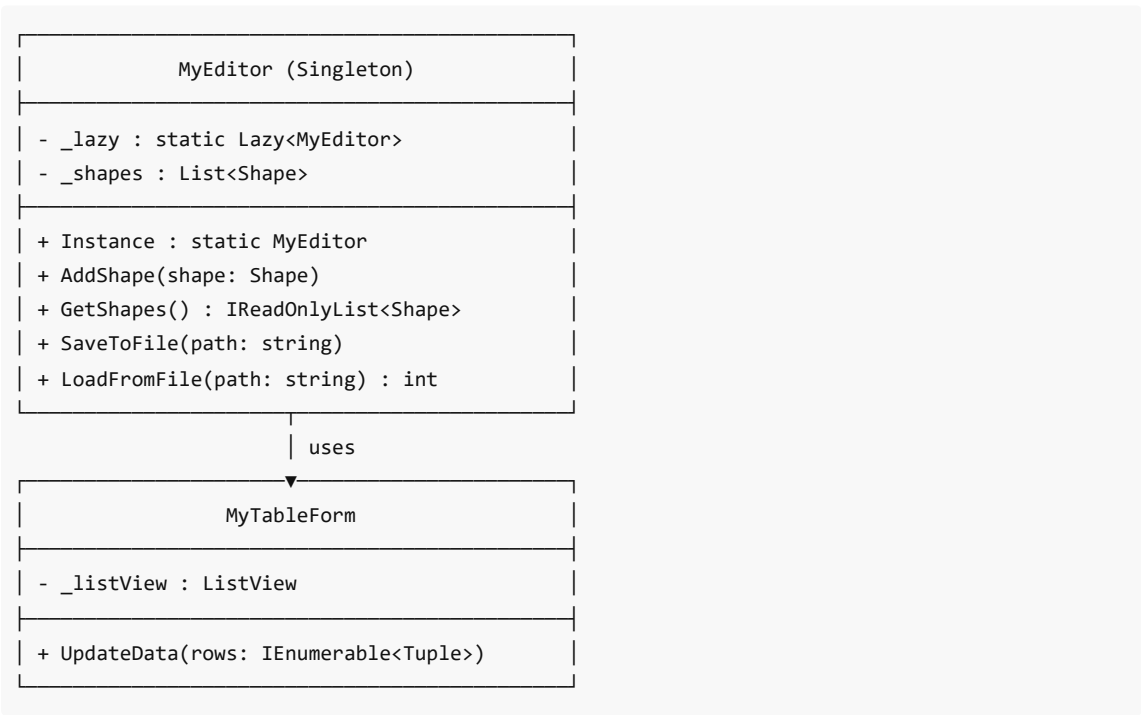
```

Діаграми

Діаграма залежностей модулів



Діаграма класів (UML) з Singleton



Скріншоти

Головне вікно та немодальне вікно таблиці

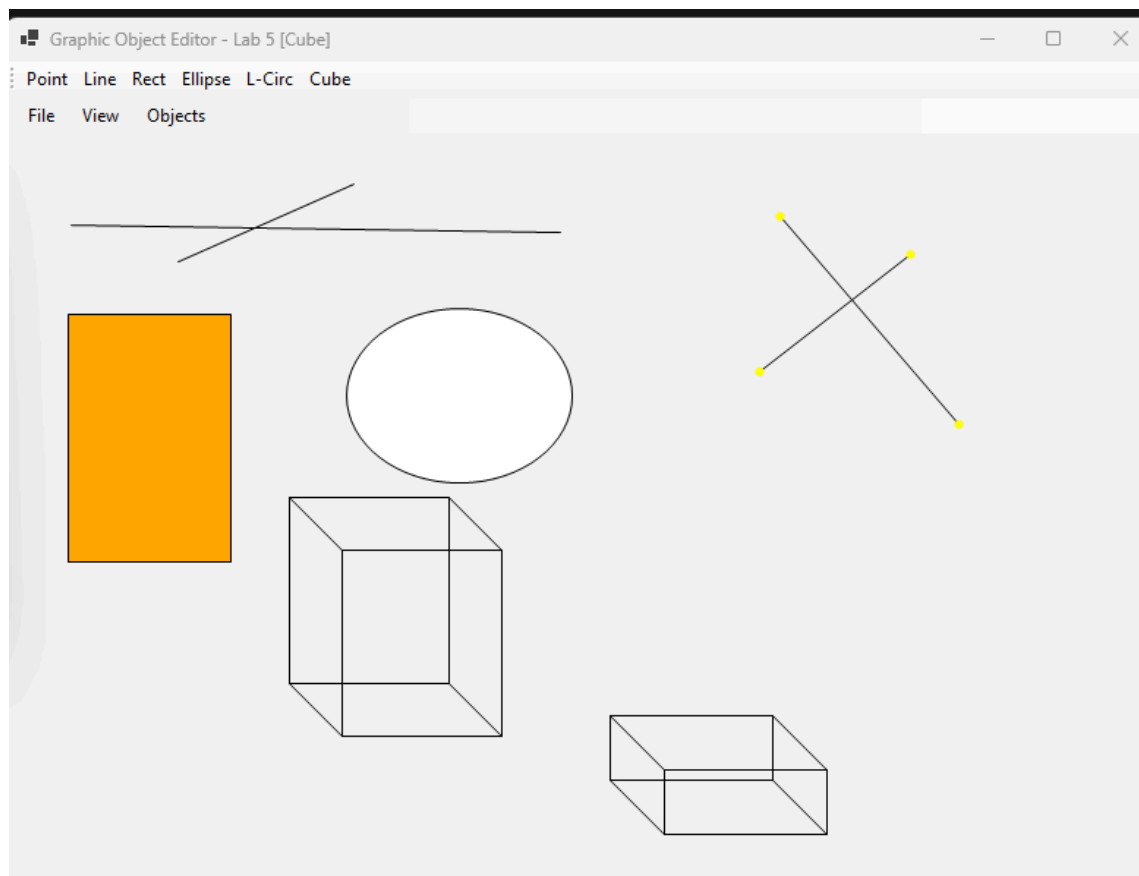


Рис. 1. Одночасна робота головного вікна редактора та немодальної таблиці об'єктів

Збереження об'єктів у файл

Shapes Table				
Name	X1	Y1	X2	Y2
Line	44	111	378	116
Line	237	83	117	136
Rectangle	42	172	153	341
Ellipse	232	168	386	287
LineWithCircles	617	131	514	211
LineWithCircles	528	105	650	247
CubeWireframe	193	297	302	424
CubeWireframe	523	446	412	490

Рис. 2. Діалог збереження координат створених об'єктів у текстовий файл.

Висновки

У цій лабораторній роботі реалізовано багатовіконний інтерфейс користувача з підтримкою збереження стану програми у файл.

Клас `MyEditor` рефакторено в потокобезпечний **Singleton** `Meepca` через `Lazy<T>`, що виключило дублювання стану редактора. Створено незалежний немодальний модуль `MyTableForm`, який забезпечує перегляд характеристик усіх створених об'єктів та синхронно оновлює свої дані при малюванні або завантаженні об'єктів з текстового файлу.